

# Agent sessions and the Agents window

From an in-editor assistant to long-lived agent sessions

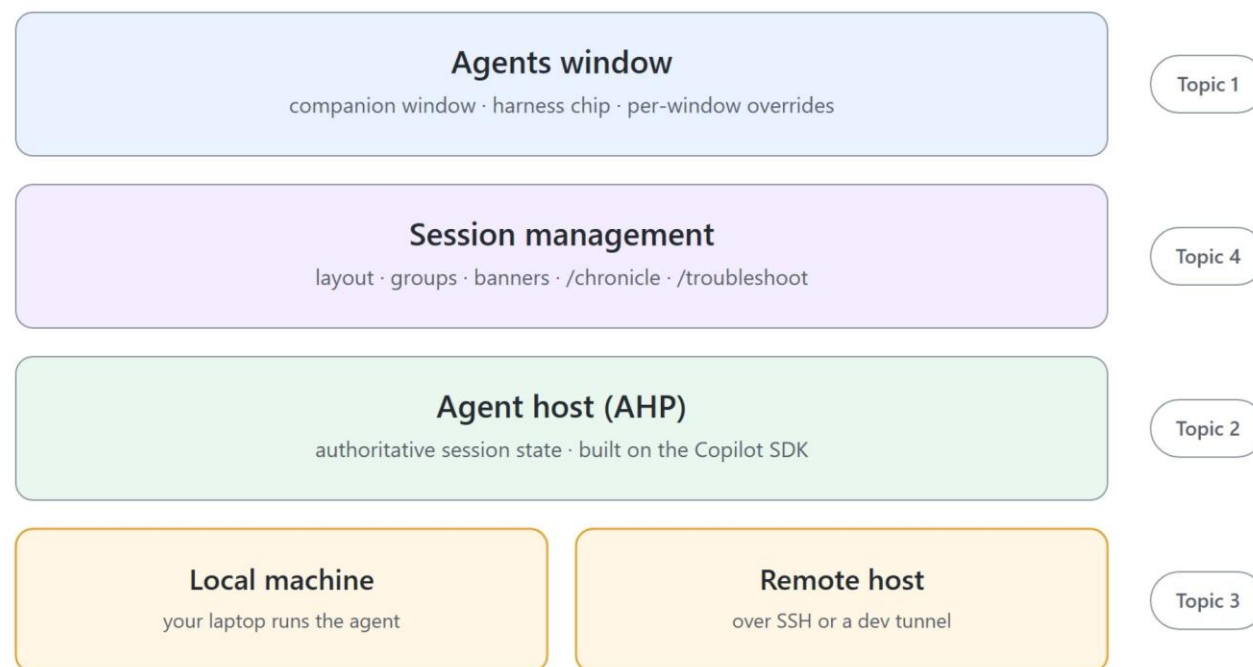
- Copilot has moved from an in-editor assistant to an **agent-session model**
- The **Agents window** is the companion surface where that model lives
- Sessions run across projects, locally or on remote hosts, as **first-class, long-lived objects**



# The session-centric product

Four topics, one stack from window to host

- **The Agents window:** a dedicated companion window with a selectable harness
- **Agent Host Protocol:** session state lives on a long-lived host, not in the client
- **Remote sessions:** the agent runs over SSH or a dev tunnel, your editor stays local
- **Session management:** side by side, groups, background send, `/chronicle`, `/troubleshoot`



# Read-first slash commands

Safe to reach for while a session is live

- `/chronicle`: query session history with `standup`, `search`, `reindex`
- `/troubleshoot`: analyze the session's own record to diagnose a stall or a failed tool call
- `/btw`: ask a side question without interrupting the running turn
- **Ctrl+R** opens the Sessions picker to jump between sessions



# The Agents window

The primary surface for agent work, beside the editor

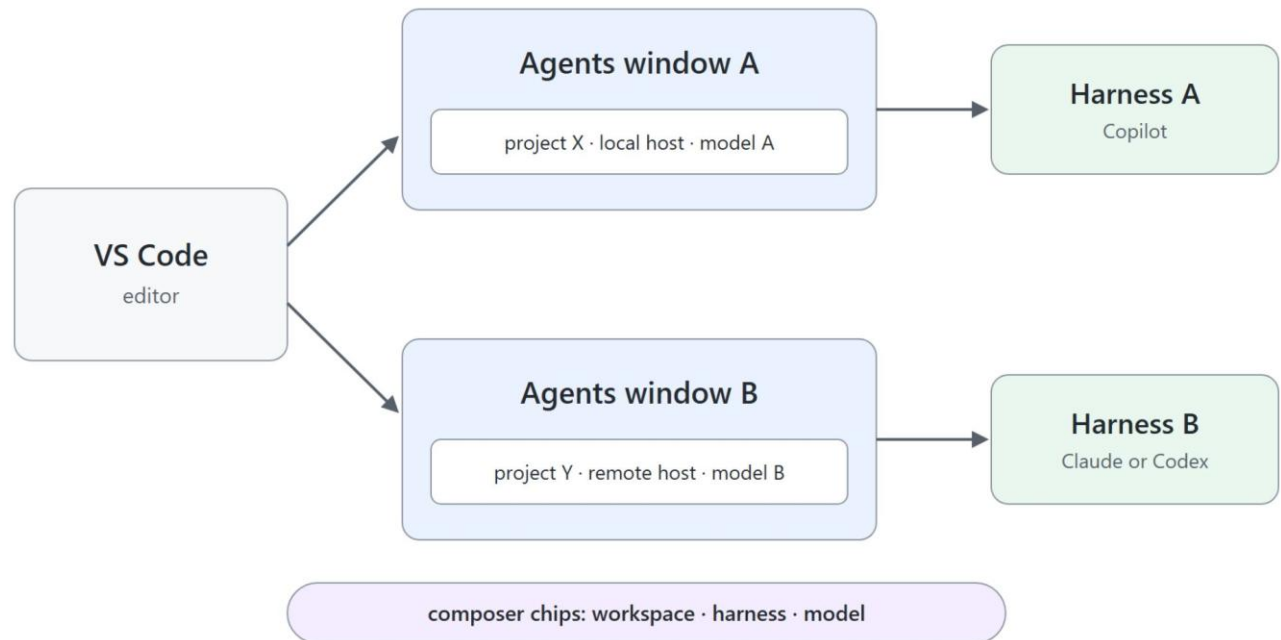
- A **companion window** for agent sessions, kept out of the main editor layout
- A **selectable harness** per window: Copilot, Claude, or Codex
- **Remote execution** and **per-window overrides** for model, project, and host
- Open it with **Open in Agents**, Chat: `Open Agents window`, or code `--agents`



# How one window differs from the next

Each window carries its own context and target

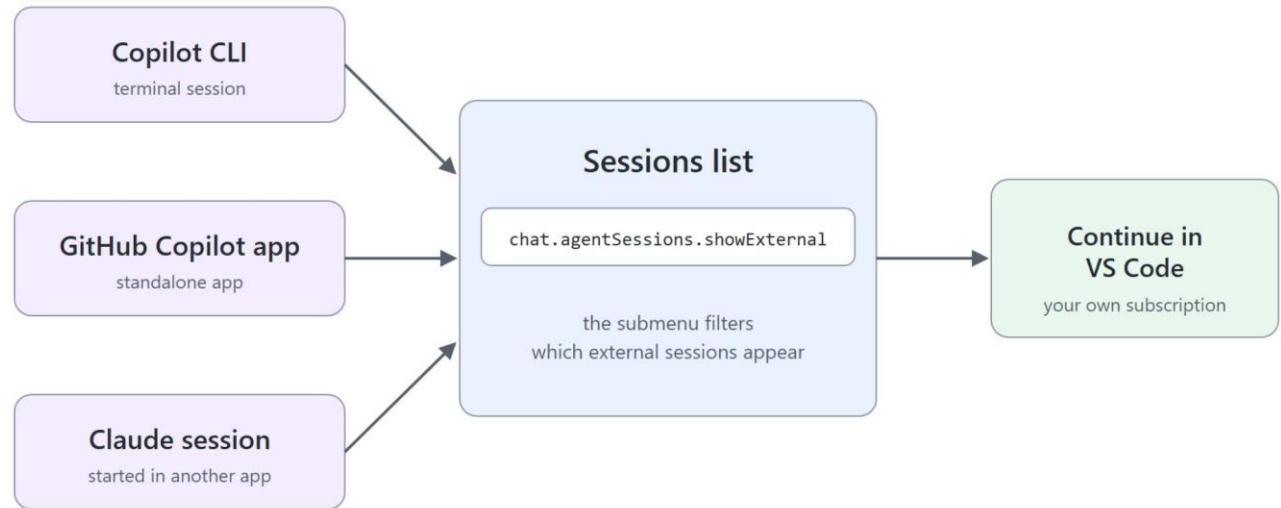
- **New** or **Ctrl+N** opens the composer with three chips: **workspace**, **harness**, **model**
- A window overriding model and host changes **no other window**
- Several projects run **at the same time** without interfering
- **Ctrl+R** walks the Sessions picker, **Ctrl+Shift+T** reopens the last closed chat



# Sessions started somewhere else

## Continue CLI and app sessions in VS Code

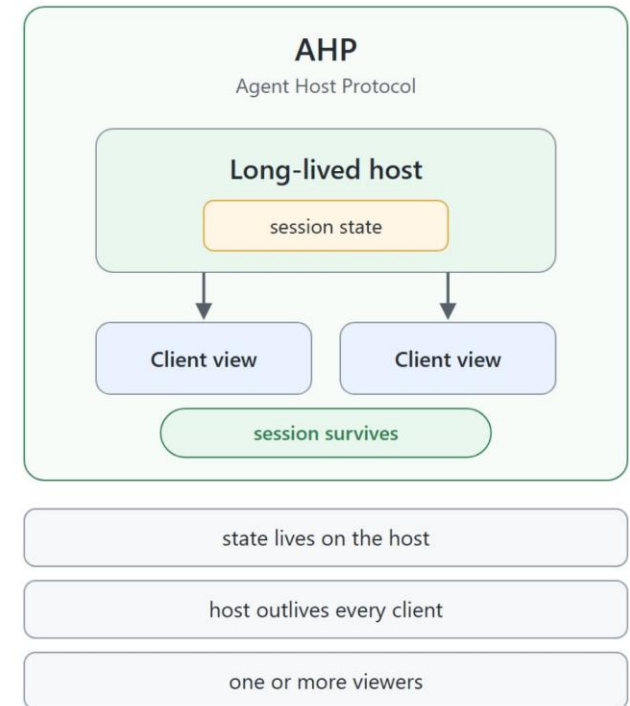
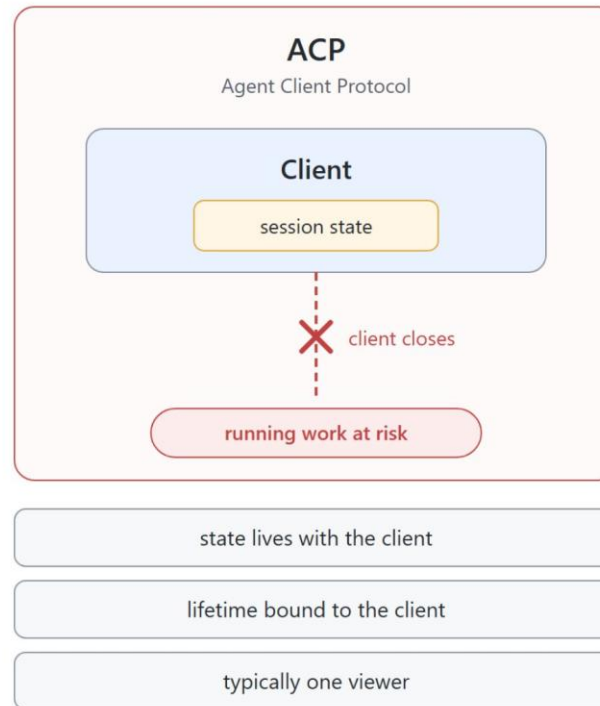
- `chat.agentSessions.showExternal` shows **Copilot or Claude sessions** from other applications
- Sources include the **Copilot CLI** and the standalone **GitHub Copilot app**
- Select one to see its conversation and **continue it** against your own subscription
- The Sessions list submenu **filters** what appears on a busy machine



# AHP vs ACP at a glance

Where session state lives decides what survives

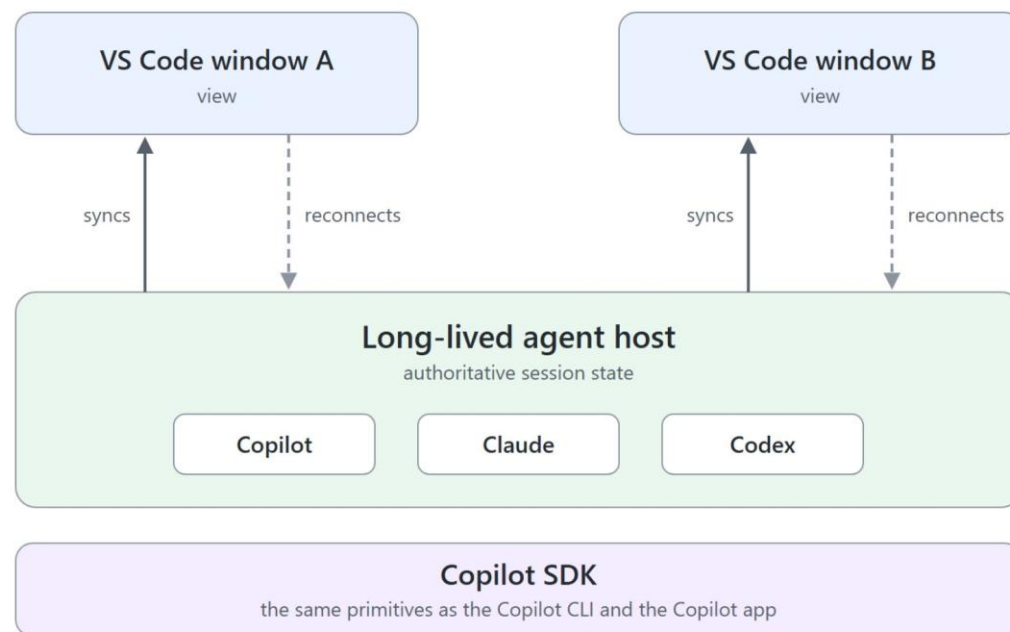
- **ACP**: authoritative state lives with the client; closing it risks the running work
- **AHP**: authoritative state lives on a **long-lived host** that outlives clients
- Under AHP the client is **a view**, and one session can have **several viewers**
- An **architect-track** topic: the boundary between where state lives and where you view it



# Host-authoritative state

Built on the Copilot SDK, shared by every harness

- Every Agents window session **already runs on the host**; nothing to switch on
- Copilot, Claude, and Codex **share the same host**
- The Copilot agent is powered by the **Copilot SDK**, matching the CLI and the Copilot app
- Settings control **which harnesses and hosts** are offered, not the protocol



# The session, not the window, owns the work

Close the window mid-run and come back

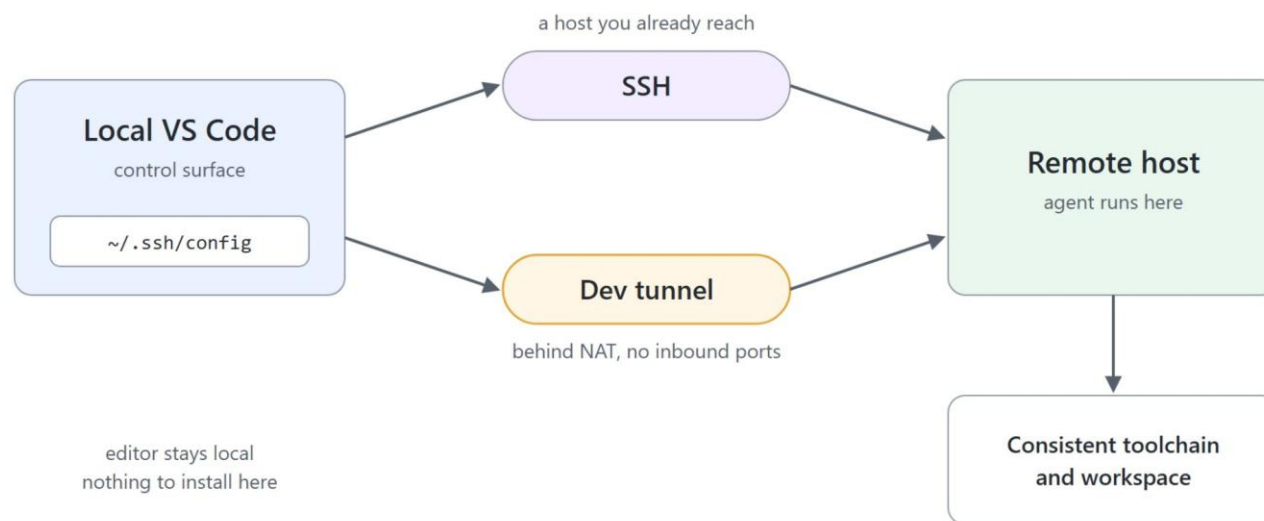
- Start a task long enough to still be running **a minute from now**
- **Close the window** while the agent works
- Reopen it: the session is **still running**, carrying the turns that completed
- Open it in a **second window**: both views track **one** session



# Remote agent sessions

SSH and dev tunnels to a reproducible host

- The agent executes on a **consistent, cloud-backed** host, not your laptop
- **SSH**: a host you already reach, whose runtimes and credentials the agent inherits
- **Dev tunnel**: hosts behind NAT or a firewall, **no inbound ports**
- Replaces the **Codespaces and Dev Container** workflow in this course



# Hello world on a Raspberry Pi

Nothing installed on the laptop

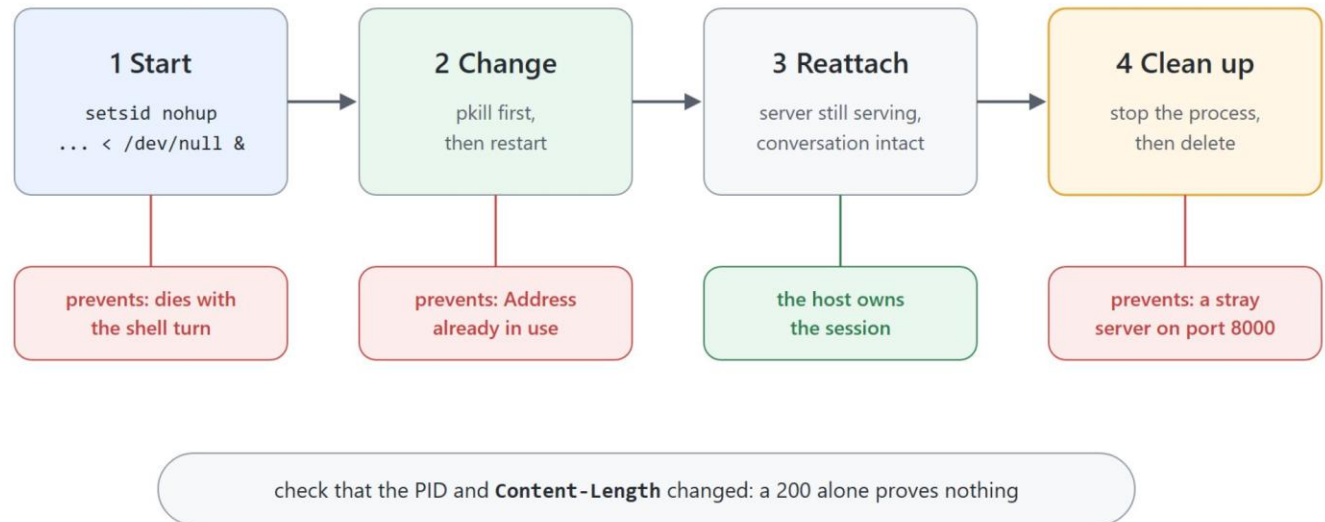
- An entry in `~/.ssh/config` makes `raspi-4` **selectable by name**; pin the key with `IdentityFile`
- Workspace chip: **Local** to **Remote** tab, pick the host, select `/home/alex`
- First connect installs the **VS Code CLI on the Pi**; nothing else
- The agent writes, runs, and curls the service **on the Pi**; port 8000 is forwarded to you



# The load-bearing details

Start detached, stop first, clean up in order

- `setsid nohup ... < /dev/null &`: a plain background start **dies with the shell turn**
- **Stop before restart**, or `Address already in use` hides in the log while old code answers
- `[p]ython3` stops `pkill` from **killing the agent's own shell**
- Clean up in order: **stop, then delete**; check that the PID and `Content-Length` changed



# Layout, navigation, and background work

Managing many sessions becomes the bottleneck

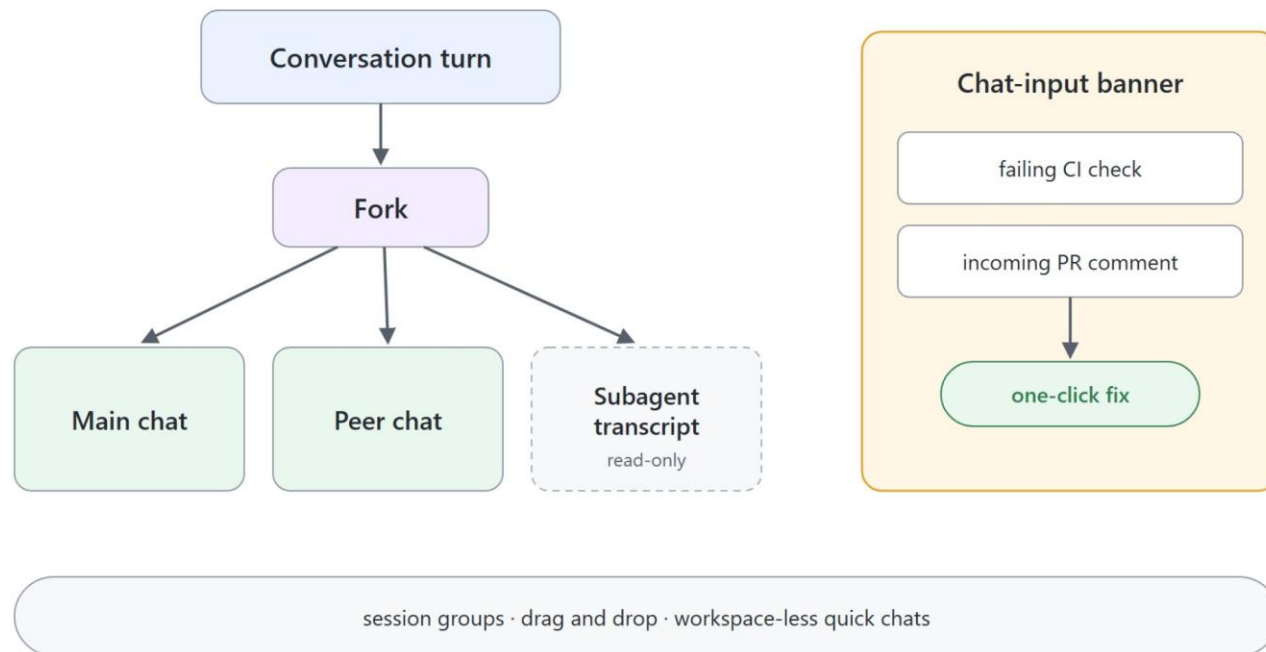
- **Side by side**, pin, and maximize keep two related sessions visible
- `Ctrl+R` Sessions picker; `Ctrl+Shift+O` jumps **within** the current session
- **Background send** keeps a session working while you move on
- **Restore on reload** keeps in-flight sessions; Close All clears them



# Groups, banners, and multi-chat

Signals and responses in the same place

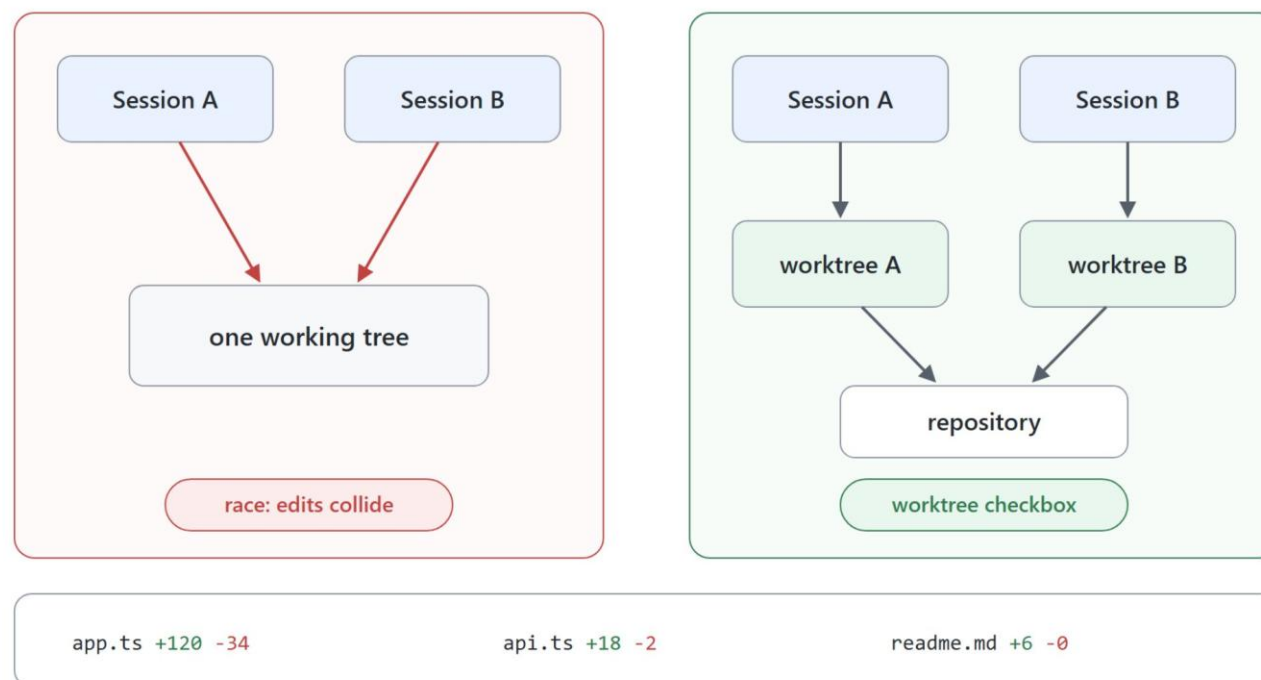
- **Session groups** with drag and drop, like organizing files
- **Chat-input banners** surface failing CI checks and PR comments with one-click actions
- **Multi-chat** forks a turn into peer chats; Claude subagent transcripts are **read-only**
- Review threads via `addComment`, `listComments`, `resolveComments`



# Isolation, diffs, and side chats

Two sessions on one working tree is a race

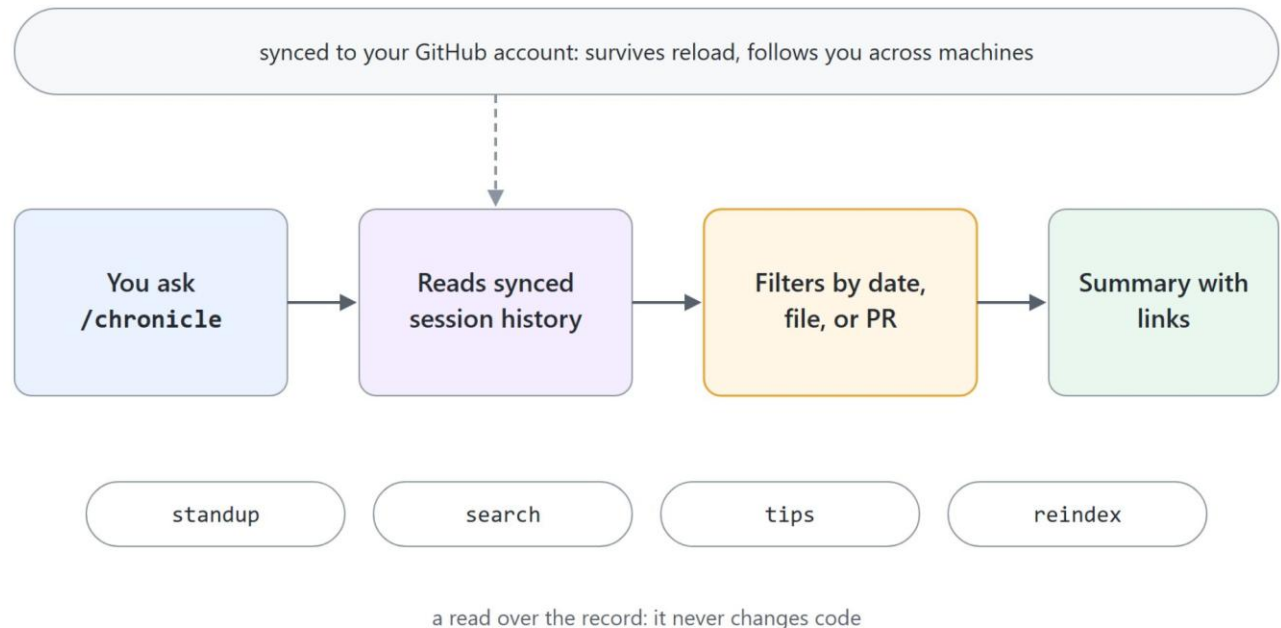
- The **worktree checkbox** puts a session in its own Git worktree, on every harness
- **File-level diff stats** and a compact multi-file diff view for long runs
- `/btw` side chats share the turn's **context and prompt cache**
- **Find in Chat** (Ctrl+F) searches collapsed work summaries too



# Session persistence and /chronicle

A searchable record of what was attempted

- Sessions **sync to your GitHub account** and survive reload
- **Cross-machine**: start on the laptop, resume on the desktop
- `/chronicle standup` and `/chronicle search` query the record, never the workspace
- `/chronicle reindex` when a query misses: **the index goes stale, not the history**



# Diagnosing a session with /troubleshoot

Name the failing step, not the error text

- Reads the **session's own record**: prompts, tool calls, failures
- Reports **what went wrong and where**, with a likely cause and fix
- For **stalls**, **failed tool calls**, and runs that end with no visible reason
- Works in local and **Copilot CLI** sessions: the diagnosis travels with the session



# Sessions are durable objects

Run many, run remote, find them again

- The **window** runs sessions side by side, the **host** owns their state
- **Remote hosts** give every session the same toolchain
- `/chronicle` and `/troubleshoot` turn history into answers

